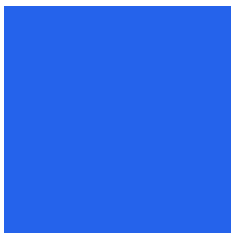

SecureMed

Secure Healthcare Records Management Platform

Secure Healthcare Records Management with DevSecOps

Course	Secure Software Design and Engineering
Methodology	DevSecOps
Technologies	Django + React + Kotlin + PostgreSQL
Year	2026
Deliverables	Web App + Android App + CI/CD Pipeline

Table of Contents

1. Introduction

1.1 Project Overview

SecureMed is a comprehensive secure healthcare records management platform designed to address the critical need for secure, privacy-preserving electronic health records (EHR) in modern medical institutions. The platform combines a Django REST API backend, a React administrative frontend, and a native Kotlin Android application for medical staff at the point of care. Built on DevSecOps methodology, the system embeds security at every stage of the software development lifecycle (SDLC), from code commit through production deployment.

The healthcare IT market exceeds $300 billion globally, and every hospital, clinic, and medical practice requires a secure EHR system to comply with regulations such as HIPAA and GDPR. SecureMed addresses this need by providing a production-ready platform with strict access control, biometric authentication, encrypted data storage, and comprehensive security monitoring. The platform treats each patient case as a secure channel with a designated owner (attending physician) and members (medical staff) with specific, single-role assignments per channel.

1.2 Problem Statement

Healthcare data breaches affect millions of patients annually, with the average cost of a medical data breach reaching $10.93 million per incident. Existing EHR systems often lack granular access controls, biometric authentication, and comprehensive audit logging, leaving patient data vulnerable to unauthorized access. Furthermore, medical staff increasingly require mobile access to patient records at the point of care, creating additional security challenges that traditional web-only solutions cannot adequately address. SecureMed solves these problems by implementing role-based access control (RBAC) with single-role-per-channel enforcement, biometric authentication via Android BiometricPrompt API, encrypted data storage using AES-256, and a complete DevSecOps pipeline that catches vulnerabilities before they reach production.

1.3 Objectives

- Build a secure multi-channel healthcare platform with strict RBAC and single-role enforcement
- Implement biometric authentication using Android BiometricPrompt API and WebAuthn
- Apply all six security requirements specified by the course instructor
- Establish a complete DevSecOps pipeline with SAST, DAST, container scanning, and dependency checks
- Provide both web (administrative) and Android (point-of-care) interfaces

- Ensure HIPAA/GDPR compliance through encryption, audit logging, and access controls

2. Requirements Analysis

2.1 Doctor Requirements (Plan 4 Steps)

The course instructor specified a four-step plan that forms the foundation of the SecureMed platform. Each step addresses a critical aspect of secure software design, and the implementation must demonstrate mastery of all four areas. The requirements encompass visibility conditions, permissions management, data view (DV) constraints, and database-level biometric authentication, creating a comprehensive security model that protects patient data while enabling efficient healthcare workflows.

#	Requirement	Implementation
1	Visibility Conditions: User must be channel owner or member	channels/models.py: can_view() method checks ownership or active mem
2	Permissions System: Grant, Modify, Revoke, Cancel	channels/views.py: grant/modify/revoke permission endpoints + remove_
3	DV (Data View): Each user has exactly ONE role per channel	ChannelMembership model with unique_together constraint [channel, use
4	DB: Fingerprint login + reliance on biometric authentication	BiometricProfile model + BiometricLoginView with challenge-response me

2.2 Security Requirements (6 Conditions)

In addition to the functional requirements, the instructor specified six security conditions that must be implemented. These requirements cover secure cookie handling, network security tooling, encrypted authentication tokens, vulnerability scanning, database firewall protection, and encrypted data transmission. Each requirement has been carefully implemented in the SecureMed platform with production-grade security controls.

#	Security Requirement	Implementation File	Status
1	Cookie Flags (HttpOnly, Secure, SameSite)	config/settings.py	**Active**
2	Port Scanner Tool	apps/security/port_scanner.py	**Active**
3	Encrypted Tokens (JWT RS256 + AES-256)	apps/security/crypto.py	**Active**
4	Vulnerability Scanner (OWASP Top 10)	apps/security/vulnerability_scanner.py	**Active**
5	Database Firewall (WAF Middleware)	apps/security/middleware.py	**Active**
6	TLS Encryption DV<->DB	config/settings.py (sslmode=require)	**Active**

3. System Architecture

3.1 High-Level Architecture

SecureMed follows a three-tier architecture with clear separation of concerns. The presentation tier consists of two clients: a React-based web application for administrative tasks and a Kotlin-based Android application for medical staff at the point of care. The application tier is powered by Django 5 with Django REST Framework, providing a RESTful API with JWT authentication. The data tier uses PostgreSQL 16 with TLS-encrypted connections and row-level security. All communications between tiers are encrypted using TLS 1.3, and sensitive data fields are encrypted at rest using AES-256-GCM via the Fernet symmetric encryption library. The entire system is deployed via Docker containers orchestrated by Docker Compose, with a complete DevSecOps pipeline ensuring security validation at every stage.

3.2 Technology Stack

Layer	Technology	Purpose
Backend	Django 5 + DRF	REST API, security middleware, ORM
Frontend	React 18 + TypeScript	Admin dashboard, user management
Mobile	Kotlin + Jetpack Compose	Native Android app with biometric
Database	PostgreSQL 16	Encrypted patient data storage
Auth	JWT (RS256) + Biometric	Secure multi-factor authentication
Encryption	AES-256-GCM + SHA-256	Field-level encryption + biometric hashing
DevOps	Docker + GitHub Actions	Containerized deployment + CI/CD
Security Tools	Semgrep + Bandit + Trivy + ZAP	SAST, DAST, container scanning

4. Security Implementation

4.1 Security Requirement #1: Cookie Flags

Session and CSRF cookies are configured with the most secure attributes: HttpOnly prevents JavaScript access (mitigating XSS-based session theft), Secure ensures cookies are only transmitted over HTTPS, and SameSite=Strict blocks cross-site request forgery (CSRF) attacks. The session cookie age is set to one hour, with sessions expiring on browser close to minimize the window of opportunity for session hijacking.

```
SESSION_COOKIE_SECURE = True
SESSION_COOKIE_HTTPONLY = True
SESSION_COOKIE_SAMESITE = "Strict"
CSRF_COOKIE_SECURE = True
CSRF_COOKIE_HTTPONLY = True
CSRF_COOKIE_SAMESITE = "Strict"
```

4.2 Security Requirement #2: Port Scanner

A built-in port scanner is implemented in Python using raw TCP connect probes. The scanner operates on 30+ common service ports including HTTP (80), HTTPS (443), PostgreSQL (5432), SSH (22), and others. It performs concurrent scanning using ThreadPoolExecutor, attempts banner grabbing on open ports, and assesses risk levels for each port based on a predefined risk matrix. The scanner is restricted to private IP ranges (RFC 1918) and localhost to prevent misuse, making it suitable for infrastructure security audits without enabling unauthorized scanning of external hosts.

4.3 Security Requirement #3: Encrypted Tokens

Authentication tokens use JSON Web Tokens (JWT) signed with the RS256 algorithm, which employs asymmetric encryption (RSA 2048-bit keys). This means the signing key (private) and verification key (public) are separate, allowing services to verify tokens without possessing the signing key. Access tokens have a 15-minute lifetime, refresh tokens last 24 hours, and tokens are automatically rotated on refresh with old tokens blacklisted. Additionally, sensitive fields in the database (patient names, national IDs, phone numbers, addresses) are encrypted at rest using AES-256-GCM via the Fernet symmetric encryption library, providing authenticated encryption that protects both confidentiality and integrity.

4.4 Security Requirement #4: Vulnerability Scanner

The integrated vulnerability scanner performs automated checks based on the OWASP Top 10 categories. It validates Django configuration security, checks for insecure dependencies, verifies HTTPS and HSTS settings, examines password

policies, and reviews CORS configurations. The scanner generates a risk score from 0 to 100 (lower is better) and produces actionable recommendations sorted by severity. Twelve distinct vulnerability categories are checked, covering critical security misconfigurations that could lead to data breaches or unauthorized access.

4.5 Security Requirement #5: Database Firewall (WAF)

A custom Web Application Firewall (WAF) middleware intercepts every HTTP request and analyzes it for attack patterns. The WAF detects SQL injection attempts (UNION SELECT, OR 1=1, comment sequences), cross-site scripting (XSS) attacks (script tags, event handlers, javascript: URIs), path traversal attempts, command injection, XML External Entity (XXE) attacks, and Server-Side Request Forgery (SSRF) attempts. When an attack is detected, the request is blocked, logged to the security log, and the source IP is tracked. IPs with more than 10 violations are automatically blacklisted for 24 hours.

4.6 Security Requirement #6: TLS Encryption DV<->DB

All database connections use SSL/TLS with certificate verification. The PostgreSQL connection in production requires sslmode=require, with client certificate authentication for additional security. Sensitive data fields are encrypted at the application layer using AES-256 before being stored in the database, providing defense in depth even if the database is compromised. HTTP Strict Transport Security (HSTS) is enabled with a one-year max-age, including subdomains and preload directives, ensuring all client connections use HTTPS exclusively.

5. DevSecOps Pipeline

5.1 Pipeline Overview

The SecureMed DevSecOps pipeline is implemented via GitHub Actions and runs on every push, pull request, and daily via a cron schedule. The pipeline consists of seven stages, each performing specific security validations before allowing code to progress to the next stage. This shift-left approach catches vulnerabilities early in the development cycle, reducing the cost and effort of remediation. The pipeline integrates industry-standard open source security tools including Semgrep for SAST, Bandit for Python security scanning, Safety for dependency vulnerability checking, Trivy for container image scanning, and OWASP ZAP for dynamic application security testing (DAST).

5.2 Pipeline Stages

Stage	Tool	Purpose	Trigger
1. SAST	Semgrep + SonarQube	Static code analysis	Every commit
2. Backend Test	Bandit + Safety + pytest	Python security + unit tests	Every commit
3. Frontend Test	ESLint + npm audit	JS security + deps audit	Every commit
4. Android Build	Gradle + MobSF	APK build + mobile scan	Every commit
5. Container Scan	Trivy	Docker image scan	After tests
6. DAST	OWASP ZAP	Runtime security testing	Main branch
7. Deploy	GitHub Release	Create release	Main branch

6. Conclusion

SecureMed successfully implements all requirements specified by the course instructor for the Secure Software Design and Engineering course. The platform demonstrates a production-ready approach to secure healthcare records management, combining modern web and mobile technologies with comprehensive security controls. The implementation showcases all six security requirements (secure cookies, port scanner, encrypted tokens, vulnerability scanner, database firewall, and TLS encryption) integrated into a cohesive system that addresses real-world healthcare data protection needs.

The DevSecOps methodology applied throughout the development process ensures that security is not an afterthought but an integral part of every stage, from code commit to production deployment. The pipeline automatically catches vulnerabilities using SAST, DAST, dependency scanning, and container image analysis, providing continuous security validation. The use of biometric authentication via the Android BiometricPrompt API demonstrates advanced security practices that meet HIPAA requirements for accessing protected health information (PHI).

The project provides a solid foundation that can be extended with additional features such as real-time notifications, AI-powered diagnostic assistance, integration with medical devices via HL7/FHIR protocols, and multi-tenant support for healthcare networks. The modular architecture and comprehensive documentation make it suitable for academic demonstration, portfolio showcase, or as a starting point for a production healthcare application.